

LAB REPORT

Title:-

CS3101: Operating System Lab 3 –System Call

Name: Parth Singla

Roll Number: 2401CS18

Objective

The primary objective of this laboratory assignment is to understand the internal architecture of the xv6 operating system by implementing custom system calls. This involves mastering the mechanisms required to cross the user-kernel space boundary safely. Specifically, the assignment aims to provide hands-on experience with:

- Registering and routing system calls through the kernel dispatcher.
- Extracting and validating arguments passed from user space using kernel helper functions.
- Safely copying structured data from isolated kernel memory back to user space.
- Traversing and reading core kernel data structures, such as the physical memory free list (`kmem`) and the process table (`proc`).
- Managing kernel-level concurrency and strictly adhering to locking protocols (`acquire` and `release`) to prevent race conditions while inspecting active processes.

Task 1:

nano kernel/syscall.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "SYS_addnumbers" kernel/syscall.h
24:#define SYS_addnumbers 23
```

nano kernel/sysproc.c

```
uint64
sys_addnumbers(void)
{
    int first, second;

    // Retrieve argument 0 and 1 (argint returns void, so we just call it directly)
    argint(0, &first);
    argint(1, &second);

    // If either argument is outside the permitted range, return -1
    if(first < 0 || first > 1000 || second < 0 || second > 1000) {
        return -1;
    }

    // Compute and return the sum of the two arguments
    return first + second;
}
```

nano kernel/syscall.c

```
extern uint64 sys_addnumbers(void);
```

```
[SYS_addnumbers]    sys_addnumbers,
```

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sys_addnumbers" kernel/syscall.c
106:extern uint64 sys_addnumbers(void);
136: [SYS_addnumbers]    sys_addnumbers,
```

nano user/user.h

```
int addnumbers(int, int);
```

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "addnumbers" user/user.h
30:int addnumbers(int, int);
```

nano user/usys.pl

```
entry("addnumbers");
```

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "addnumbers" user/usys.pl
46:entry("addnumbers");
```

nano user/addtest.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int test_case(int t_num, int a, int b, int expected) {
    int result = addnumbers(a, b);
    if (result == expected) {
        printf("T%d: addnumbers(%d, %d) returned %d, expected %d: PASS\n", t_num, a, b, result, expected);
        return 0;
    } else {
        printf("T%d: addnumbers(%d, %d) returned %d, expected %d: FAIL\n", t_num, a, b, result, expected);
        return 1;
    }
}

int main(void) {
    int failed_tests = 0;

    failed_tests += test_case(1, 20, 22, 42);
    failed_tests += test_case(2, 0, 0, 0);
    failed_tests += test_case(3, 1000, 1000, 2000);
    failed_tests += test_case(4, -1, 10, -1);
    failed_tests += test_case(5, 10, 1001, -1);

    if (failed_tests > 0) {
        printf("addtest: %d TESTS FAILED\n", failed_tests);
        exit(1);
    }

    printf("addtest: ALL TESTS PASSED\n");
    exit(0);
}
```

nano Makefile

```
$U/_addtest
```

```
parthsingla@172-9-4-40 xv6-riscv % grep -n "_addtest" Makefile
151:     $U/_addtest
```

make clean

```
parthsingla@172-9-4-40 xv6-riscv % make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
    */*.o */*.d */*.asm */*.sym \
    kernel/kernel fs.img \
    mkfs/mkfs .gdbinit \
    user/usys.S \
    user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls
user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombi
e user/_logstress user/_forphan user/_dorphan user/_sync user/_sysinfotest user/_procinfotest
user/_addtest
```

make qemu

```
xv6 kernel is booting
```

```
hart 2 starting
hart 1 starting
init: starting sh
$ □
```

addtest

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ addtest
T1: addnumbers(20, 22) returned 42, expected 42: PASS
T2: addnumbers(0, 0) returned 0, expected 0: PASS
T3: addnumbers(1000, 1000) returned 2000, expected 2000: PASS
T4: addnumbers(-1, 10) returned -1, expected -1: PASS
T5: addnumbers(10, 1001) returned -1, expected -1: PASS
addtest: ALL TESTS PASSED
$ █
```

Task 2:

nano kernel/sysinfo.h

```
#ifndef _SYSINFO_H_
#define _SYSINFO_H_

struct sysinfo {
    uint64 freemem; // amount of free memory (bytes)
    uint64 nproc;  // number of process
};

#endif
```

nano kernel/kalloc.c

```
uint64
count_free_mem(void)
{
    struct run *r;
    uint64 total_bytes = 0;

    // Acquire the lock to ensure safe reading while other processes allocate
    acquire(&kmem.lock);

    r = kmem.freelist;
    while(r) {
        total_bytes += 4096; // PGSIZE in xv6 is 4096 bytes
        r = r->next;
    }

    release(&kmem.lock);
    return total_bytes;
}
```

nano kernel/proc.c

```
uint64
count_active_processes(void)
{
    struct proc *p;
    uint64 count = 0;

    for(p = proc; p < &proc[NPROC]; p++) {
        // Acquire the lock for the specific process before reading its state
        acquire(&p->lock);
        if(p->state != UNUSED) {
            count++;
        }
        // Release the lock immediately after checking
        release(&p->lock);
    }

    return count;
}
```

nano kernel/defs.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -nE "count_free_mem|count_active_processes" kernel/defs.h
63:uint64          count_free_mem(void);
107:uint64         count_active_processes(void);
○ parthsingla@172-9-4-40 xv6-riscv %
```

nano kernel/sysproc.c

```
uint64
sys_sysinfo(void)
{
    uint64 user_dst;
    struct sysinfo info;
    struct proc *p = myproc();

    // Retrieve argument 0 as a user-space destination address
    argaddr(0, &user_dst);

    // Store the available-memory and active-process counts in the local structure
    info.freemem = count_free_mem();
    info.nproc = count_active_processes();

    // Safely copy the structure from kernel space to user space
    if(copyout(p->pagetable, user_dst, (char *)&info, sizeof(info)) < 0)
        return -1; // Return -1 on failure

    return 0; // Return 0 on success
}
```

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sysinfo.h" kernel/sysproc.c
9:#include "sysinfo.h"
```

nano kernel/syscall.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "SYS_sysinfo" kernel/syscall.h
25:#define SYS_sysinfo 24
```

nano kernel/syscall.c

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sys_sysinfo" kernel/syscall.c
107:extern uint64 sys_sysinfo(void);
138: [SYS_sysinfo] sys_sysinfo
```

nano user/user.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sysinfo" user/user.h
2:#include "kernel/sysinfo.h"
31:int sysinfo(struct sysinfo *);
```

nano user/usys.pl

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sysinfo" user/usys.pl
47:entry("sysinfo");
```

nano user/sysinfotest.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/sysinfo.h"
#include "user/user.h"

int main(void) {
    struct sysinfo info;
    int pfd[2]; // File descriptors for the pipe

    if (sysinfo(&info) < 0) {
        printf("sysinfotest: FAIL (sysinfo returned error)\n");
        exit(1);
    }

    printf("Before fork: free memory = %d bytes, processes = %d\n", (int)info.freemem, (int)info.nproc);

    // Create a pipe for synchronization
    if (pipe(pfd) < 0) {
        printf("sysinfotest: pipe failed\n");
        exit(1);
    }

    int pid = fork();
    if (pid < 0) {
        printf("sysinfotest: fork failed\n");
        exit(1);
    }

    if (pid == 0) {
        // CHILD PROCESS: Block and wait for the parent to write to the pipe
        char buf;
        read(pfd[0], &buf, 1);
        exit(0);
    } else {
        // PARENT PROCESS: Check info while child is blocked
        if (sysinfo(&info) < 0) {
            printf("sysinfotest: FAIL\n");
            exit(1);
        }
        printf("During fork: free memory = %d bytes, processes = %d\n", (int)info.freemem, (int)info.nproc);

        // Write a dummy byte to the pipe to release the child
        write(pfd[1], "x", 1);
        wait(0);
    }

    printf("sysinfotest: OK\n");
    exit(0);
}
```

nano Makefile

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "_sysinfotest" Makefile
149:     $U/_sysinfotest\
```

make clean

```
● parthsingla@172-9-4-40 xv6-riscv % make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
    /*.o /*.d /*.asm /*.sym \
    kernel/kernel fs.img \
    mkfs/mkfs .gdbinit \
    user/usys.S \
    user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls
user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombi
e user/_logstress user/_forphan user/_dorphan user/_sync user/_sysinfotest user/_procinfotest
user/_addtest
```

make qemu sysinfotest

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$ sysinfotest
```

```
Before fork: free memory = 133263360 bytes, processes = 3
```

```
During fork: free memory = 133218304 bytes, processes = 4
```

```
sysinfotest: OK
```

```
$ █
```

Task 3:

nano kernel/procinfo.h

```
#ifndef _PROCINFO_H_
```

```
#define _PROCINFO_H_
```

```
struct procinfo {
    int pid;          // Process ID
    int state;        // Process state (numeric)
    uint64 memory;    // Virtual memory size in bytes
    int open_files;   // Number of open file descriptors
};
```

```
#endif
```

nano kernel/proc.c

```
#include "procinfo.h"

int get_procinfo(int pid, struct procinfo *info)
{
    struct proc *p;
    int retry = 0;

    do {
        retry = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);

            if(p->state == UNUSED || p->pid != pid) {
                release(&p->lock);
                continue;
            }

            // If the target is actively running on another CPU, it's unsafe to copy
            if(p->state == RUNNING && p != myproc()) {
                release(&p->lock);
                retry = 1;
                break;
            }

            // We found the process and hold its lock safely
            info->pid = p->pid;
            info->state = p->state;
            info->memory = p->sz; // Size in bytes

            // Count open file descriptors without assuming a fixed number
            int fd_count = 0;
            for(int fd = 0; fd < NOFILE; fd++) {
                if(p->ofile[fd]) {
                    fd_count++;
                }
            }
            info->open_files = fd_count;

            release(&p->lock);
            return 0; // Success
        }

        if(retry) {
            yield(); // Give the running process time, then try again
        }
    } while(retry);

    return -1; // Process not found
}
```

nano kernel/defs.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "get_procinfo" kernel/defs.h
109:int get_procinfo(int, struct procinfo*);
```

nano kernel/sysproc.c

```
uint64
sys_procinfo(void)
{
    int pid;
    uint64 user_dst;
    struct procinfo info;
    struct proc *p = myproc();

    // Retrieve argument 0 (pid) and argument 1 (user address)
    argint(0, &pid);
    argaddr(1, &user_dst);

    // Validate the user-space destination address basic boundary check
    if(user_dst == 0)
        return -1;

    // Locate the process and fill our local kernel structure
    if(get_procinfo(pid, &info) < 0) {
        return -1; // Invalid or nonexistent PID
    }

    // Safely copy from kernel space to user space
    if(copyout(p->pagetable, user_dst, (char *)&info, sizeof(info)) < 0)
        return -1;

    return 0;
}
```

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "procinfo.h" kernel/sysproc.c
10:#include "procinfo.h"
```

nano kernel/syscall.h

```
201#include "procinfo.h"
● parthsingla@172-9-4-40 xv6-riscv % grep -n "SYS_procinfo" kernel/syscall.h
26:#define SYS_procinfo 25
```

nano kernel/syscall.c

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "sys_procinfo" kernel/syscall.c
108:extern uint64 sys_procinfo(void);
137: [SYS_procinfo] sys_procinfo,
```

nano user/user.h

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "procinfo(" user/user.h
32:int procinfo(int, struct procinfo *);
```

nano user/usys.pl

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "procinfo" user/usys.pl
48:entry("procinfo");
```

nano user/procinfotest.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fcntl.h"
#include "kernel/procinfo.h"
#include "user/user.h"

int main(void) {
    struct procinfo info;
    int my_pid = getpid();

    // 1. Test current process
    if(procinfo(my_pid, &info) < 0) exit(1);
    printf("Current process:\nPID = %d\nState = %d\nMemory = %d bytes\nOpen files = %d\n",
        info.pid, info.state, (int)info.memory, info.open_files);

    // 2. Test file descriptor changes
    int fd = open("README", O_RDONLY);
    procinfo(my_pid, &info);
    printf("After open:\nOpen files = %d\n", info.open_files);
    close(fd);
    procinfo(my_pid, &info);
    printf("After close:\nOpen files = %d\n", info.open_files);

    // 3. Test invalid PID and pointers
    if(procinfo(-999, &info) != -1) printf("Failed to catch invalid PID\n");
    if(procinfo(my_pid, (struct procinfo *)0) != -1) printf("Failed to catch invalid pointer\n");

    // 4. Test child process with pipe synchronization
    int pfd[2];
    pipe(pfd);
    int cpid = fork();

    if(cpid == 0) {
        char buf;
        read(pfd[0], &buf, 1); // Child blocks here
        exit(0);
    } else {
        // Let child run and block using a delay loop to bypass the missing sleep syscall
        for(volatile int i = 0; i < 5000000; i++);

        procinfo(cpid, &info);
        printf("Child process:\nPID = %d\nState = %d\nMemory = %d bytes\nOpen files = %d\n",
            info.pid, info.state, (int)info.memory, info.open_files);

        write(pfd[1], "x", 1); // Release child
        wait(0);
    }

    printf("procinfotest: ALL TESTS PASSED\n");
    exit(0);
}
```

nano Makefile

```
● parthsingla@172-9-4-40 xv6-riscv % grep -n "_procinfotest" Makefile
150:    $U/_procinfotest\
```

```
make clean
make qemu
procinfotest
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ procinfotest
Current process:
PID = 3
State = 4
Memory = 16384 bytes
Open files = 3
After open:
Open files = 4
After close:
Open files = 3
Child process:
PID = 4
State = 3
Memory = 16384 bytes
Open files = 5
procinfotest: ALL TESTS PASSED
$
```

Results and Discussion

The assignment was successfully executed across three progressively complex tasks, with all edge cases and synchronization requirements met.

- **Task 1 (`addnumbers`):** The foundational system call was successfully integrated into the xv6 kernel. The implementation successfully utilized `argint` to fetch user arguments from trap frames. The output confirmed that the system call accurately computed sums for valid inputs while strictly enforcing boundary conditions (rejecting negative values and values exceeding 1000).
- **Task 2 (`sysinfo`):** This task successfully demonstrated the extraction of global system statistics. By traversing the `kmem` linked list and the `proc` array, the system accurately tracked available physical memory and active process counts. The use of `copyout` successfully bridged the user-kernel memory divide to populate a shared `sysinfo` struct. The execution results clearly illustrated the dynamic allocation of memory and the consumption of a process slot immediately following a `fork()`. Furthermore, strict compiler constraints (`-Werror`) regarding 64-bit type casting were successfully navigated.
- **Task 3 (`procinfo`):** The final task successfully implemented a targeted process inspection tool, requiring complex concurrency management. The implementation accurately locked individual process table entries before reading sensitive state data, memory sizes, and open file descriptors. During testing, the open file count dynamically shifted as expected when opening and closing files. To properly test child process inheritance and state (`SLEEPING` vs. `RUNNING`), a user-space synchronization strategy utilizing pipe blocking (and subsequently a busy-wait delay loop to bypass linking constraints) was successfully employed. The output verified that the child process correctly inherited standard file descriptors along with the synchronization pipe.

Conclusion

This laboratory exercise provided profound, hands-on experience with the internal operations of a Unix-like kernel. Implementing `addnumbers`, `sysinfo`, and `procinfo` demystified the execution flow of system calls, from user-space assembly stubs to kernel-space handlers. It highlighted the critical importance of memory isolation, safe data-copying techniques, and rigorous concurrency control when designing and modifying operating systems. Ultimately, this assignment established a strong, practical foundation in systems programming and kernel architecture.

